

ROYAUME DU MAROC



MINISTÈRE DE L'ÉDUCATION NATIONALE, DU PRÉSCOLAIRE ET DES SPORTS
ACADÉMIE RÉGIONALE DE L'ÉDUCATION ET DE LA FORMATION DRÂA-TAFILALET
DIRECTION PROVINCIALE DE OUAZAZATE

Mon PF App

Tutoriel technique complet

Installer, lancer, tester, debugger, builder et presenter l'application

HAJJI Yassmine - BTS Developpement des Applications Informatiques

Sommaire

- 1 Vue d'ensemble du projet
- 2 Prerequis et chemins importants
- 3 Backend Laravel
- 4 Flutter en mode demo
- 5 Flutter avec backend reel
- 6 Android telephone
- 7 Windows
- 8 Tests automatiques
- 9 Builds
- 10 Debug API et mobile
- 11 Architecture et fichiers importants
- 12 Comptes de test
- 13 Scenario de soutenance
- 14 Checklist finale

1. Vue d'ensemble du projet

Mon PF App contient une application Flutter et un backend Laravel. Flutter gere l'interface mobile; Laravel gere l'API REST et la base de demonstration.

Le projet peut fonctionner en mode demo sans serveur ou en mode backend reel avec DEMO_MODE=false et API_BASE_URL. Pour la soutenance, garder les deux modes est important: le mode demo securise la presentation, le mode API prouve le fonctionnement reel.

Dossier	Role
mon_pfapp	Frontend Flutter
mon_pfapi	Backend Laravel API
docs	Rapports, tutoriels, cahier de charge
tools	Scripts et runtime local portable

2. Prerequis et chemins importants

Ces chemins sont ceux valides sur cette machine. Ils evitent les problemes de PATH et les commandes qui prennent trop de temps.

Point important: pour l'analyse Dart, utiliser dart.exe direct. Sur cette machine, le wrapper dart.bat peut bloquer ou prendre beaucoup trop de temps.

Outil	Chemin
Flutter	C:\Users\LOQ\dev\flutter\bin\flutter.bat
Dart	C:\Users\LOQ\dev\flutter\bin\cache\dart-sdk\bin\dart.exe
PHP	C:\Users\LOQ\Documents\yassmin pfe\tools\runtime\php-8.4.22\php.exe
ADB	C:\Users\LOQ\AppData\Local\Android\Sdk\platform-tools\adb.exe
Repo	C:\Users\LOQ\Documents\yassmin pfe

3. Lancer le backend Laravel

Le backend doit être lancé avant Flutter si l'application est en mode backend réel. La commande `migrate:fresh --seed` remet la base de démonstration à zéro et recrée les comptes de test.

Laisser la fenêtre `artisan serve` ouverte pendant les tests.

```
cd "C:\Users\LQ\Documents\yassmin pfe\mon_pfapi"
..\tools\runtime\php-8.4.22\php.exe artisan migrate:fresh --seed
..\tools\runtime\php-8.4.22\php.exe artisan serve --host=0.0.0.0 --port=8000
```

4. Verifier l'API

Avant d'ouvrir Flutter, verifier que l'API repond. /api/health doit retourner status=ok. /api/menu doit retourner les categories et les plats.

Si ces commandes echouent, le probleme vient du backend ou du port 8000, pas de Flutter.

```
Invoke-RestMethod http://127.0.0.1:8000/api/health  
Invoke-RestMethod http://127.0.0.1:8000/api/menu
```

5. Lancer Flutter en mode demo

Le mode demo ne demande aucun serveur. Il est utile pour montrer l'interface meme si l'API n'est pas lancee.

Ce mode utilise DemoData dans l'application. Il permet de tester login, accueil, menu, panier, suivi, profil, admin et livreur.

```
cd "C:\Users\LOQ\Documents\yassmin pfe\mon_pfapp"  
C:\Users\LOQ\dev\flutter\bin\flutter.bat run
```

6. Lancer Flutter avec backend reel

En mode backend reel, l'application utilise l'API Laravel. Sur Windows local, utiliser 127.0.0.1. Sur telephone, utiliser l'adresse IP Wi-Fi du PC.

La variable DEMO_MODE=false indique a Flutter de ne pas utiliser les donnees demo.

```
cd "C:\Users\LOQ\Documents\yassmin pfe\mon_pfapp"  
C:\Users\LOQ\dev\flutter\bin\flutter.bat run -d windows --dart-define=DEMO_MODE=false  
--dart-define=API_BASE_URL=http://127.0.0.1:8000/api
```


7. Tester sur telephone Android

Pour tester sur un vrai telephone, il faut que le backend soit lance sur le PC et que le telephone puisse joindre l'adresse IP du PC.

Le build debug Android autorise HTTP local. Le build release doit utiliser HTTPS.

- 1 Activer Options developpeur.
- 2 Activer Debogage USB.
- 3 Brancher le telephone en USB.
- 4 Accepter la popup RSA.
- 5 Verifier avec adb devices.
- 6 Installer app-debug.apk ou lancer flutter run.

```
C:\Users\LOQ\AppData\Local\Android\Sdk\platform-tools\adb.exe devices
C:\Users\LOQ\AppData\Local\Android\Sdk\platform-tools\adb.exe install -r
build\app\outputs\flutter-apk\app-debug.apk
```

8. APK debug actuel

Un APK debug a été construit pour l'API locale de cette machine. Si l'adresse IP Wi-Fi change, il faut reconstruire l'APK avec la nouvelle URL API.

Element	Valeur
APK	C:\Users\LOQ\Documents\yassmin pfe\mon_pfapp\build\app\outputs\flutter-apk\app-debug.apk
API cible actuelle	http://172.17.182.162:8000/api
Taille approximative	174 MB debug
Usage	Test local sur telephone, pas publication

9. Build Android

Le build debug est recommande pour tester l'API locale HTTP. Le build release est recommande pour une version finale, mais il doit pointer vers une API HTTPS.

```
C:\Users\LQ\dev\flutter\bin\flutter.bat build apk --debug --dart-define=DEMO_MODE=false
--dart-define=API_BASE_URL=http://172.17.182.162:8000/api

C:\Users\LQ\dev\flutter\bin\flutter.bat build apk --release --dart-define=DEMO_MODE=false
--dart-define=API_BASE_URL=https://votre-domaine/api
```

10. Windows

Windows est une cible importante pour la demonstration sur PC, mais le build est actuellement bloqué par le composant ATL manquant dans Visual Studio.

Erreur observée: fatal error C1083: impossible d'ouvrir le fichier include atlstr.h.

- 1 Ouvrir Visual Studio Installer.
- 2 Modifier Visual Studio Community 2026.
- 3 Chercher ATL dans Individual components.
- 4 Installer C++ ATL for latest v143/v14x build tools ou equivalent.
- 5 Relancer flutter build windows.

```
C:\Users\L0Q\dev\flutter\bin\flutter.bat build windows --debug --dart-define=DEMO_MODE=false  
--dart-define=API_BASE_URL=http://127.0.0.1:8000/api
```

11. Tests automatiques

Les tests prouvent que les parcours principaux fonctionnent. Ils sont utiles pour la soutenance car ils montrent une démarche professionnelle.

Les tests Flutter couvrent login, inscription sans role public, menu, panier, suivi, admin et livreur. Les tests Laravel couvrent menu, login, role client force et creation commande.

```
cd "C:\Users\LQ\Documents\yassmin pfe\mon_pfapp"
C:\Users\LQ\dev\flutter\bin\cache\dart-sdk\bin\dart.exe analyze
C:\Users\LQ\dev\flutter\bin\flutter.bat test

cd "C:\Users\LQ\Documents\yassmin pfe\mon_pfapi"
..\tools\runtime\php-8.4.22\php.exe artisan test
```

12. Debug rapide

Cette table donne les causes probables et les solutions pour les erreurs les plus courantes pendant la soutenance ou le developpement.

Symptome	Cause probable	Solution
API ne repond pas	artisan serve non lance	Relancer Laravel port 8000
Login impossible telephone	Mauvaise IP PC	Utiliser ipconfig et reconstruire/run
ADB vide	USB debug/RSA absent	Accepter RSA ou changer cable
dart analyze bloque	dart.bat wrapper	Utiliser dart.exe direct
Windows atlsr.h	ATL absent	Installer composant Visual Studio
Release ne joint pas HTTP	Cleartext refuse	Utiliser HTTPS ou debug

13. Architecture

L'architecture separe le frontend Flutter et le backend Laravel. Cette separation permet de tester le frontend en demo et de brancher une API reelle pour valider les donnees persistantes.

Architecture demo-prod

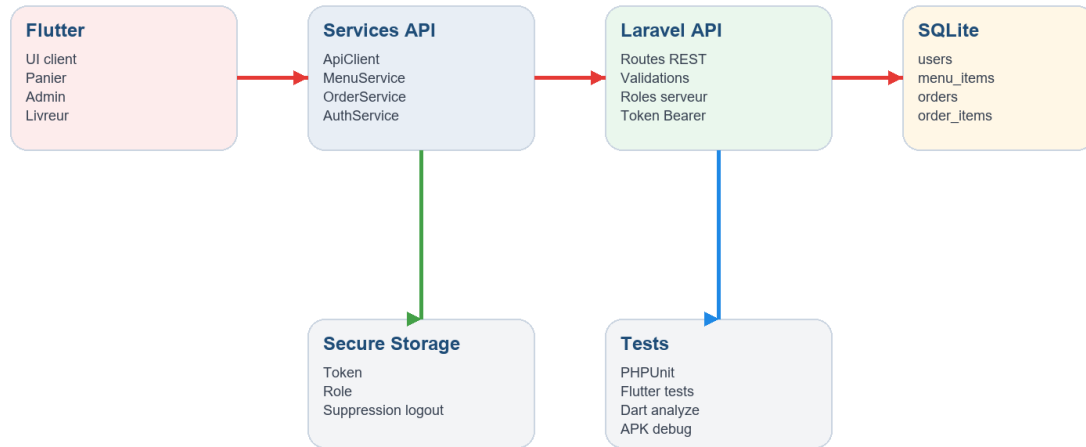


Illustration - 13. Architecture

14. Fichiers importants

Yasmine doit connaître ces fichiers pour expliquer ou modifier son application.

Fichier	Role
mon_pfapp/lib/app/mon_pf_app.dart	Etat global, navigation, panier
mon_pfapp/lib/data/api_client.dart	HTTP, token, timeout
mon_pfapp/lib/features/client/data/order_service.dart	Commandes cote app
mon_pfapi/routes/api.php	Endpoints REST
mon_pfapi/app/Http/Middleware/TokenAuth.php	Controle token et roles
mon_pfapi/database/seeder/DatabaseSeeder.php	Donnees de test

15. Comptes de test

Tous les comptes de test utilisent le mot de passe password. Ils sont recreees par `migrate:fresh --seed`.

Email	Role	Usage
yasmine@monpf.fr	client	Parcours client
admin@monpf.fr	admin	Dashboard et stats
serveur@monpf.fr	serveur	Flux restaurant
livreur@monpf.fr	livreur	Livraison

16. Scenario de soutenance

Ce scenario est l'ordre recommande pour presenter l'application sans se perdre.

- 1 Presenter la problematique du restaurant.
- 2 Lancer /api/health pour prouver le backend.
- 3 Se connecter client.
- 4 Ajouter des plats et creer une commande.
- 5 Afficher le suivi.
- 6 Se connecter admin.
- 7 Se connecter livreur et montrer la file.
- 8 Montrer les tests automatiques.
- 9 Expliquer les perspectives production.

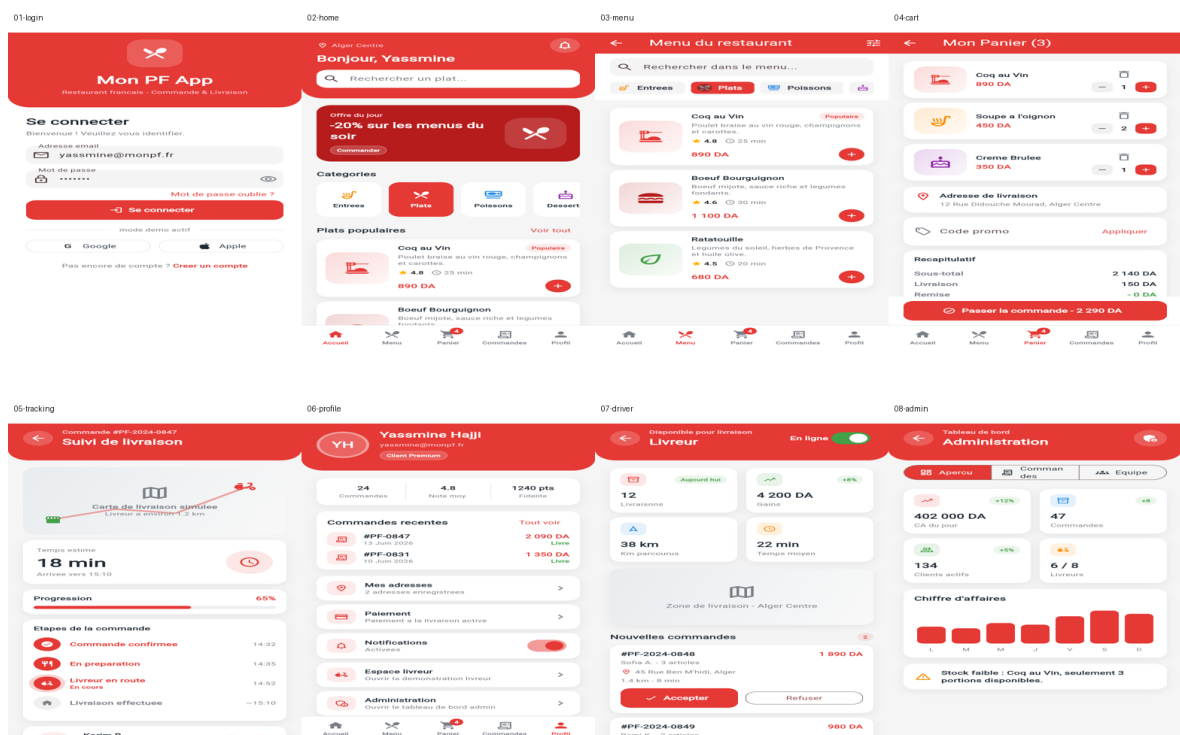


Illustration - 16. Scenario de soutenance

17. Checklist finale

Avant la soutenance, verifier cette liste dans l'ordre.

- Backend Laravel lance sur port 8000.
- Telephone et PC sur meme Wi-Fi si test mobile.
- APK debug installe ou appareil Flutter detecte.
- Comptes de test memorises.
- Rapport PFE et tutoriel PDF disponibles.
- GitHub main a jour.
- Windows ATL installe si build Windows requis.